

Code -

```
#include <iostream>
#include <vector>
#include <chrono>
#include <omp.h>
using namespace std;

void bubbleSortSequential(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

void bubbleSortParallel(vector<int>& arr) {
    int n = arr.size();

    for (int i = 0; i < n; i++) {
        #pragma omp parallel for
        for (int j = 0; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }

        #pragma omp parallel for
        for (int j = 1; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }
}

int main() {
    int N;

    cout << "Enter the size of the vector: ";
    cin >> N;

    vector<int> arr(N), original(N);

    cout << "Enter the elements of the vector (space-separated): ";
    for (int i = 0; i < N; i++) {
        cin >> arr[i];
        original[i] = arr[i];
    }

    auto start = chrono::high_resolution_clock::now();
    bubbleSortSequential(arr);
    auto end = chrono::high_resolution_clock::now();

    cout << "Sequential Bubble Sort Time (ms): "
         << chrono::duration<double>(end - start).count() << endl;

    arr = original;

    start = chrono::high_resolution_clock::now();
```

```
bubbleSortParallel(arr);  
end = chrono::high_resolution_clock::now();  
  
cout << "Parallel Bubble Sort Time (ms): "  
      << chrono::duration<double>(end - start).count() << endl;  
  
return 0;  
}
```

Output -

```
Enter the size of the vector: 5  
Enter the elements of the vector (space-separated): 5 3 1 4 2  
Sequential Bubble Sort Time (ms): 6.3e-07  
Parallel Bubble Sort Time (ms): 7.2e-07
```

=== Code Execution Successful ===